

Angular Performance and Scalability

Techniques, code examples, and a real-world production challenge

Angular 18 . TypeScript 5.4 . CDK 18 . RxJS 7.8 . May 2025

1. The Core Principle

Angular performance comes down to one question: **how much work does the framework do on each user interaction?**

Every technique in this guide reduces unnecessary work -- fewer change detection cycles, smaller bundles, less DOM manipulation, fewer re-renders for unchanged data.

The order of attack matters. Profile first with Chrome DevTools, fix the highest-impact issue, measure again, then repeat.

2. Change Detection -- The Biggest Lever

The problem with the default strategy

With **ChangeDetectionStrategy.Default**, Angular walks the entire component tree on every browser event -- a button click, a scroll, a keypress. In a dashboard with 180+ components, that is hundreds of redundant checks per second.

```
// Every component checked on every event
@Component({
  changeDetection: ChangeDetectionStrategy.Default
  // checks all 200 components on every single click
})
```

Fix 1: OnPush strategy

OnPush tells Angular to skip a component unless its @Input reference changes, an Observable/Signal emits, or markForCheck() is called manually.

```

@Component({
  selector: 'app-claims-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `...`
})
export class ClaimsListComponent {
  @Input() claims!: Claim[];
  // Only re-checks when this reference changes
}

```

Fix 2: Angular 18 Signals -- fine-grained reactivity

Signals go one step further than OnPush. Only the exact template expression that reads a signal re-evaluates -- not even the full component template. This is "zoneless" Angular -- no Zone.js watching every async operation.

```

dashboard.component.ts
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <p>Count: {{ count() }}</p>           // re-renders ONLY when count changes
    <p>Doubled: {{ doubled() }}</p>       // re-renders ONLY when count changes
    <p>Name: {{ userName() }}</p>        // re-renders ONLY when userName changes
  `
})
export class DashboardComponent {
  count = signal(0);
  userName = signal('Suresh');

  // Computed -- auto-tracks dependencies, memoised
  doubled = computed(() => this.count() * 2);

  // Effect -- side effects when signals change
  private logEffect = effect(() => {
    console.log('count changed:', this.count());
  });

  increment() {
    this.count.update(n => n + 1);
    // Only {{ count() }} and {{ doubled() }} re-render
  }
}

```

Signals vs BehaviorSubject

	BehaviorSubject	Signal
Read	.value or subscribe()	call as function: count()
Write	.next(value)	.set(value) or .update(fn)

	BehaviorSubject	Signal
Derived	.pipe(map(...))	computed(() => ...)
Template	async pipe or subscribe	{{ count() }} directly
Cleanup	must unsubscribe()	automatic
Change detection	triggers zone -- whole tree	fine-grained -- affected expressions only

toSignal -- convert Observable to Signal

```
@Component({
  template: `
    @if (users()) {
      @for (user of users()!; track user.id) {
        <p>{{ user.name }}</p>
      }
    } @else {
      <p>Loading...</p>
    }
  `
})
export class UserListComponent {
  private http = inject(HttpClient);
  // toSignal subscribes automatically + unsubscribes on destroy
  users = toSignal(this.http.get<User[]>('/api/users'));
}
```

3. Lazy Loading -- Reduce Initial Bundle

Route-level lazy loading

```

app.routes.ts
// app.routes.ts
export const routes: Routes = [
  // Eager -- always in main bundle
  { path: 'home', component: HomeComponent },

  // Lazy -- downloaded ONLY when user visits /dashboard
  {
    path: 'dashboard',
    loadComponent: () =>
      import('./dashboard/dashboard.component')
        .then(m => m.DashboardComponent)
  },

  // Lazy feature routes -- downloads entire feature lazily
  {
    path: 'claims',
    loadChildren: () =>
      import('./claims/claims.routes')
        .then(m => m.CLAIMS_ROUTES)
  }
];

```

Template-level lazy loading with @defer

@defer splits a component into a separate chunk that only downloads when the trigger fires -- e.g. when the user scrolls to it.

```

// Chart.js (200 KB) downloaded ONLY when user scrolls to this section
@defer (on viewport) {
  <app-claims-chart [data]="chartData()" />
} @placeholder {
  <div style="height: 400px; background: #f5f5f5; border-radius: 8px;">
    Chart loads when visible
  </div>
} @loading (minimum 300ms) {
  <app-skeleton-chart />
} @error {
  <p>Chart failed to load.</p>
}

```

@defer trigger types

Trigger	When it fires
on viewport	When the placeholder scrolls into view
on interaction	On first click or keypress inside the placeholder
on idle	When the browser has spare CPU time
on timer(2000)	After a 2-second delay

Trigger	When it fires
on hover	On mouse enter
when condition	When a signal/expression becomes truthy

4. TrackBy in @for -- Prevent Full DOM Rebuild

Without track, Angular has no way to identify which item changed. On any data refresh it destroys and recreates every DOM node in the list.

```
// Without track -- SLOW: full DOM rebuild on every refresh
@for (claim of claims; track claim) {
  <app-claim-row [claim]="claim" />
}

// With track claim.id -- FAST: only changed items re-render
@for (claim of claims; track claim.id) {
  <app-claim-row [claim]="claim" />
}
```

Angular 18 enforces track -- the compiler gives an error if you omit it. It is mandatory, not optional.

5. Virtual Scrolling -- Handle Huge Lists

Without virtual scrolling, 2,400 claim rows creates 2,400 DOM nodes -- each participating in every change detection cycle. CDK Virtual Scroll renders only ~15 visible rows regardless of list size.

```
claims-list.component.ts
import { ScrollingModule } from '@angular/cdk/scrolling';

@Component({
  standalone: true,
  imports: [ScrollingModule],
  template: `
    <cdk-virtual-scroll-viewport itemSize="56" style="height: 600px">
      <div *cdkVirtualFor="let claim of claims"
        class="claim-row"
        style="height: 56px;">
        <span>{{ claim.date }}</span>
        <span>{{ claim.provider }}</span>
        <span>{{ claim.amount | currency }}</span>
      </div>
    </cdk-virtual-scroll-viewport>
  `
})
export class ClaimsListComponent {
  claims: Claim[] = []; // can be 10,000+ items
}
// Result: 2,400 items -> 15 DOM nodes rendered at any time
```

6. Pure Pipes -- Cache Expensive Transforms

Template methods are called on every change detection cycle -- even when values have not changed. Pure pipes are cached and only recalculate when input reference changes.

```
// BAD: method called on every CD cycle -- hundreds of times per second
{{ formatAmount(claim.amount) }} // formatAmount() runs constantly

// GOOD: pure pipe -- cached until claim.amount changes
@Pipe({ name: 'gbpCurrency', pure: true, standalone: true })
export class GbpCurrencyPipe implements PipeTransform {
  private formatter = new Intl.NumberFormat('en-GB', {
    style: 'currency', currency: 'GBP'
  });
  transform(amount: number): string {
    return this.formatter.format(amount);
  }
}

// In template: {{ claim.amount | gbpCurrency }}
// Only called when claim.amount reference changes
```

7. RxJS -- Prevent Memory Leaks and Excessive Calls

```

@Component({ ... })
export class SearchComponent {
  private destroyRef = inject(DestroyRef);
  results = signal<SearchResult[]>([]);

  ngOnInit() {
    this.searchInput.valueChanges.pipe(
      debounceTime(300),           // wait 300ms after last keystroke
      distinctUntilChanged(),     // skip if value is same as before
      switchMap(query =>          // cancels previous HTTP request
        this.searchService.search(query).pipe(
          catchError(() => of([]))
        )
      ),
      takeUntilDestroyed(this.destroyRef) // auto-unsubscribes on destroy
    ).subscribe(results => this.results.set(results));
  }
}

```

Key operators and what they prevent

Operator	Problem it solves
<code>debounceTime(300)</code>	Fires 1 API call per pause, not 1 per keystroke
<code>distinctUntilChanged()</code>	Skips requests when value is identical to previous
<code>switchMap()</code>	Cancels in-flight HTTP request when new value arrives
<code>takeUntilDestroyed()</code>	Prevents memory leaks -- auto-unsubscribes on destroy
<code>shareReplay(1)</code>	Multiple subscribers share one HTTP call instead of duplicates
<code>forkJoin()</code>	Fires multiple API calls in parallel -- saves N x latency

8. Bundle Optimisation

```

# Full optimised production build
ng build --configuration production
# --aot: pre-compiles templates (no runtime compiler shipped)
# --optimization: tree-shakes unused code + minifies
# --build-optimizer: removes Angular decorators from output

# Analyse the bundle to find what is taking KB
npm install -g source-map-explorer
ng build --source-map
source-map-explorer dist/my-app/*.js

```

Dynamic import for large third-party libraries

```
// BAD: static import -- Chart.js always bundled (200 KB)
import Chart from 'chart.js/auto';

// GOOD: dynamic import -- only bundled when function is called
async loadChart() {
  const { Chart } = await import('chart.js/auto');
  new Chart(this.canvas.nativeElement, this.chartConfig);
}
```

9. Real-World Challenge -- CareFirst Member Portal

Context

The FEP Blue member dashboard displayed claims history, benefit statements, and financial totals for BlueCross BlueShield contract holders. With some accounts having 3+ years of claims data, the dashboard was taking **8-12 seconds** to become interactive. Users were abandoning the page before it finished loading.

First step: profile with Chrome DevTools flame chart -- not guessing. This showed 60% of main thread time consumed by change detection and 25% by DOM insertion.

Root causes found

Problem	Impact
Default CD on 180+ components	Every scroll event triggered a full tree check
Claims table rendering 2,400+ rows on load	2,400 DOM nodes in every CD cycle
formatCurrency() method in template	Called hundreds of times per second
3 API calls blocking serially	Each waited for the previous before starting
Chart.js (200 KB) in main bundle	Downloaded even on pages with no charts
No track on claim rows	Full DOM rebuild on every filter change

Fix 1: OnPush + Signals on the heaviest components

```
// Before: Default CD -- checked on every event
@Component({ changeDetection: ChangeDetectionStrategy.Default })
export class DashboardComponent {
  claims: Claim[] = []; // plain array -- no reactivity signal
}

// After: OnPush + Signals -- checked only when signal emits
@Component({ changeDetection: ChangeDetectionStrategy.OnPush })
export class DashboardComponent {
  claims = signal<Claim[]>([]);
  total = computed(() =>
    this.claims().reduce((sum, c) => sum + c.amount, 0)
  );
  // total re-computes ONLY when claims changes
}
```

Fix 2: CDK Virtual Scroll on the claims list

```
// Before: 2,400 DOM nodes created on page load
@for (claim of claims; track claim.id) {
  <app-claim-row [claim]="claim" />
}

// After: only ~15 DOM nodes rendered at any time
<cdk-virtual-scroll-viewport itemSize="56" style="height: 500px">
  <div *cdkVirtualFor="let claim of claims$">
    <app-claim-row [claim]="claim" />
  </div>
</cdk-virtual-scroll-viewport>
```

Fix 3: @defer on charts

```
// Before: Chart.js in main bundle -- always downloaded
import Chart from 'chart.js/auto';

// After: @defer -- 0 KB until user scrolls to chart section
@defer (on viewport) {
  <app-claims-chart [data]="chartData()" />
} @placeholder {
  <div style="height: 320px"></div>
}
```

Fix 4: Parallel API calls

```
// Before: serial -- 3 x 400ms = 1,200ms minimum
const claims    = await this.claimsService.getClaims();
const benefits  = await this.benefitsService.getBenefits();
const profile   = await this.profileService.getProfile();

// After: parallel with forkJoin -- ~400ms regardless
forkJoin({
  claims:    this.claimsService.getClaims(),
  benefits:  this.benefitsService.getBenefits(),
  profile:   this.profileService.getProfile()
}).subscribe(({ claims, benefits, profile }) => {
  this.claims.set(claims);
  this.benefits.set(benefits);
  this.profile.set(profile);
});
```

Results measured in production

Metric	Before	After	Improvement
Time to interactive	8-12 seconds	1.4 seconds	83% faster
Initial bundle size	~1.8 MB	~760 KB	58% smaller
Change detection cycles/sec	~1,200/sec	~215/sec	82% fewer
DOM nodes on load	2,400 nodes	15 nodes	99% fewer
Page abandonment rate	~44%	~11%	76% reduction

10. Quick Reference

Highest impact -- fix first

Technique	When to use	Expected gain
OnPush + Signals	All components with complex templates	50-80% fewer CD cycles
Lazy loading routes	Any route not needed on initial load	30-60% smaller initial bundle
CDK Virtual Scroll	Lists with 200+ items	90-99% fewer DOM nodes

Medium impact -- fix next

Technique	When to use	Expected gain
@defer (on viewport)	Charts, heavy widgets, below-fold content	20-40% smaller bundle

Technique	When to use	Expected gain
Pure pipes	Any method called in a template	Eliminates redundant calculations
track in @for	All lists with mutable data	No full DOM rebuild on update
debounceTime + switchMap	Search inputs, filter controls	Eliminates excess API calls
forkJoin	Multiple independent API calls	Parallel -- saves N x latency

Angular 18 performance features

Feature	What it does
signal()	Fine-grained reactive state -- only affected expressions re-render
computed()	Derived signals -- memoised, recalculates only when dependencies change
toSignal()	Converts Observable to Signal -- auto-subscribes and unsubscribes
@defer	Template-level lazy loading with 6 trigger types
@for with track	Required -- compiler errors if omitted
Standalone components	Fine-grained tree shaking -- only imported APIs in bundle
takeUntilDestroyed()	No-boilerplate subscription cleanup on component destroy

Angular Performance and Scalability . Angular 18 . TypeScript 5.4 . CDK 18 . RxJS 7.8 . May 2025